

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

A.

Servlet-Based Student Registration System

A college student registration system can be implemented as a dynamic web application using HTML/DOM-based client-side scripting and a Java Servlet for server-side processing. The browser collects student information through a form, sends an HTTP request to the servlet, and the servlet processes the data and generates a response.

1. Architecture of a Servlet-Based Application

The application follows a client–server architecture:

Components

1. Client/browser

- Displays the registration form.
- HTML is used to create the form.

- CSS provides presentation.
 - JavaScript/DOM can perform client-side validation and dynamically modify the page.
2. **Servlet Container**
 - Example: Apache Tomcat.
 - Receives HTTP requests.
 - Creates and manages servlet objects.
 - Calls servlet life-cycle methods.
 3. **Java Servlet**
 - Processes GET and POST requests.
 - Retrieves form parameters.
 - Performs validation and business logic.
 - Sends an HTTP response.
 4. **Database**
 - Stores student registration information.
 - Examples: MySQL, PostgreSQL, or Oracle.

2. Student Registration Form

```

<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>

  <script>
    function validateForm() {
      let name = document.getElementById("name").value;
      let email = document.getElementById("email").value;

      if (name.trim() === "" || email.trim() === "") {
        alert("Name and Email are required");
        return false;
      }

      return true;
    }
  </script>
</head>

<body>

<h2>Online Student Registration</h2>

<form action="StudentServlet" method="post"
  onsubmit="return validateForm()">

  <label>Student Name:</label>
  <input type="text" id="name" name="name"><br><br>

  <label>Register Number:</label>

```

```

<input type="text" name="regno"><br><br>

<label>Email:</label>
<input type="email" id="email" name="email"><br><br>

<label>Department:</label>
<select name="department">
    <option value="CSE">Computer Science</option>
    <option value="ECE">Electronics</option>
    <option value="EEE">Electrical</option>
    <option value="MECH">Mechanical</option>
</select><br><br>

<label>Password:</label>
<input type="password" name="password"><br><br>

<input type="submit" value="Register">

</form>

</body>
</html>

```

3. Java Servlet Implementation

```

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;
import java.io.PrintWriter;

@WebServlet("/StudentServlet")
public class StudentServlet extends HttpServlet {

    @Override
    public void init() throws ServletException {
        System.out.println("StudentServlet initialized");
    }

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        processRequest(request, response, "GET");
    }
}

```


The servlet retrieves form values using:

```
String name = request.getParameter("name");  
String email = request.getParameter("email");
```

A GET request sends parameters through the URL.

For example:

StudentServlet?name=Ganesh®no=101&email=student@example.com

Advantages of GET

- Simple to use.
- Suitable for retrieving information.
- Parameters can be bookmarked.
- Useful for search and query operations.

Disadvantages

- Data appears in the URL.
- Limited URL length.
- Sensitive information can be exposed through browser history, logs, bookmarks, and referrer information.

Therefore, GET should not be used for passwords or other sensitive registration information.

5. Handling POST Method

The POST method is handled using:

```
protected void doPost(HttpServletRequest request,  
                        HttpServletResponse response)  
    throws ServletException, IOException
```

The form specifies:

```
<form action="StudentServlet" method="post">
```

The servlet receives the data:

```
String name = request.getParameter("name");  
String regno = request.getParameter("regno");  
String email = request.getParameter("email");  
String department = request.getParameter("department");
```

Unlike GET, the form parameters are sent in the HTTP request body rather than being appended to the URL.

Advantages of POST

- Suitable for submitting form data.

- Does not expose parameters directly in the URL.
- Can handle larger amounts of data.
- Appropriate for operations that modify server-side data.

However, **POST by itself does not encrypt data**. For sensitive information such as passwords, the application should use **HTTPS/TLS**.

6. Servlet Life Cycle

The servlet life cycle is controlled by the **servlet container**, such as Tomcat.

init()

The `init()` method is called **once** when the servlet is initialized.

```
@Override
public void init() throws ServletException {
    System.out.println("StudentServlet initialized");
}
```

It can be used for:

- Loading configuration.
- Initializing resources.
- Establishing required connections.

service()

The servlet container invokes the `service()` method for every incoming request.

`HttpServlet` internally dispatches the request to the appropriate method.

For example:

```
GET request → service() → doGet()
POST request → service() → doPost()
```

In our implementation, we override `doGet()` and `doPost()` rather than directly overriding `service()`.

destroy()

The `destroy()` method is called **once** before the servlet is removed from service.

```
@Override
public void destroy() {
    System.out.println("StudentServlet destroyed");
}
```

It can be used to:

- Close database connections.
- Release resources.
- Perform cleanup operations.

8. Which Method Is Appropriate for Sensitive Data?

POST is more appropriate than GET for sensitive student registration data.

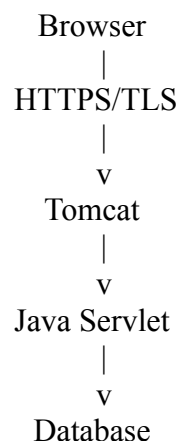
For example, the following information should be submitted using POST:

- Password
- Personal details
- Student identification information
- Other confidential registration information

The form should therefore use:

```
<form action="StudentServlet" method="post">
```

However, POST **does not provide encryption by itself**. For real-world deployment, the application should use:



Additional security measures should include:

- HTTPS/TLS.
- Server-side input validation.
- Password hashing rather than storing plaintext passwords.
- Prepared statements when using a database.
- Authentication and authorization.
- Session management.
- CSRF protection where appropriate.

Conclusion

A servlet-based student registration application uses **HTML/DOM and JavaScript on the client side** and **Java Servlets on the server side**. The servlet container manages the servlet life cycle through `init()`, request processing through `service()/doGet()/doPost()`, and cleanup

through `destroy()`. For student registration, **POST is the appropriate method for submitting sensitive information**, combined with HTTPS for actual confidentiality and integrity.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

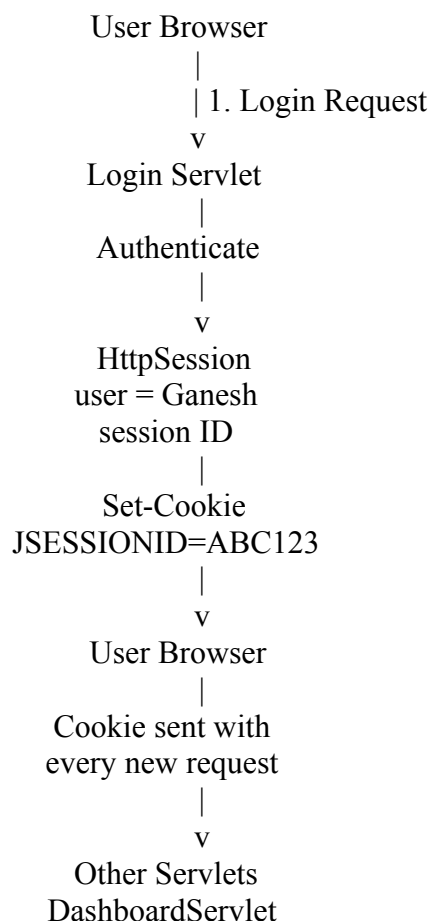
Design a session management system using `HttpSession` and `Cookies` in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

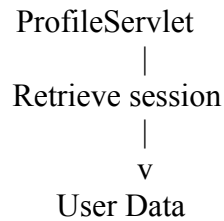
A.

Session Management Using `HttpSession` and `Cookies` in Servlets

A website that requires users to log in and access multiple pages needs **session management**. Since HTTP is stateless, the server does not automatically remember that a user has logged in. Servlet session tracking mechanisms such as **`HttpSession`** and **cookies** can be used to maintain the user's identity across multiple requests.

1. Session Management Architecture





2. How Session Tracking Works

HTTP is **stateless**. For example:

Request 1 → Login
Request 2 → Dashboard
Request 3 → Profile

Without session tracking, the server treats these as independent requests.

Session tracking creates an identifier that allows the server to associate all these requests with the same user.

A typical flow is:

1. User enters username/password
↓
2. LoginServlet validates credentials
↓
3. Server creates HttpSession
↓
4. Server assigns Session ID
↓
5. Session ID is sent to browser
↓
6. Browser sends Session ID with future requests
↓
7. Servlet retrieves the corresponding session
↓
8. User remains authenticated

3. Login Form

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
</head>

<body>

<h2>User Login</h2>
```

```

<form action="LoginServlet" method="post">

    <label>Username:</label>
    <input type="text" name="username" required>
    <br><br>

    <label>Password:</label>
    <input type="password" name="password" required>
    <br><br>

    <input type="submit" value="Login">

</form>

</body>
</html>

```

4. Login Servlet Using HttpSession

The `HttpSession` object stores information associated with a particular user.

LoginServlet.java

```

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        // Example authentication
        if (username.equals("student") &&
            password.equals("12345")) {

            HttpSession session = request.getSession();

            session.setAttribute("username", username);

```

```

        session.setAttribute("loggedIn", true);

        response.sendRedirect("DashboardServlet");

    } else {

        response.setContentType("text/html");
        response.getWriter().println(
            "<h3>Invalid username or password</h3>"
        );
    }
}
}

```

After successful authentication:

```
HttpSession session = request.getSession();
```

creates or retrieves a session.

User information is stored using:

```
session.setAttribute("username", username);
```

5. Accessing the Session on Another Page

Suppose the user opens the dashboard.

DashboardServlet.java

```

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;
import java.io.PrintWriter;

@WebServlet("/DashboardServlet")
public class DashboardServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        response.setContentType("text/html");
    }
}

```

```

        PrintWriter out = response.getWriter();

        if (session == null ||
            session.getAttribute("loggedIn") == null) {

            response.sendRedirect("login.html");
            return;
        }

        String username =
            (String) session.getAttribute("username");

        out.println("<html>");
        out.println("<body>");

        out.println("<h2>Welcome, " + username + "</h2>");
        out.println("<p>You are successfully logged in.</p>");

        out.println("<a href='ProfileServlet'>Profile</a><br>");
        out.println("<a href='LogoutServlet'>Logout</a>");

        out.println("</body>");
        out.println("</html>");
    }
}

```

The important statement is:

```
HttpSession session = request.getSession(false);
```

false means:

Get the existing session, but do not create a new session if one doesn't exist.

This is useful for protecting pages that require authentication.

6. Using Cookies for Session Tracking

Servlet containers commonly use a cookie named:

JSESSIONID

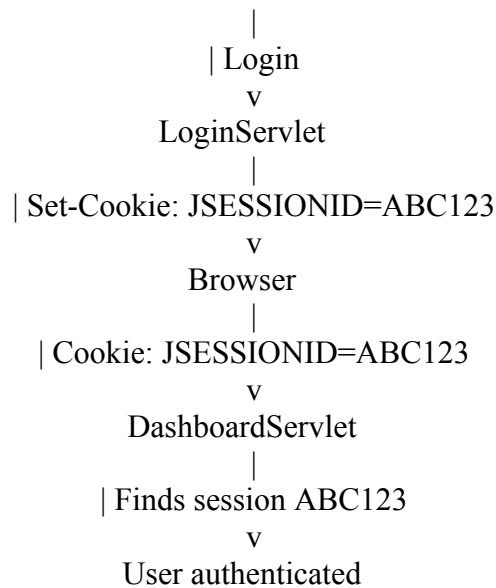
Example:

JSESSIONID=AB12CD34EF56

The browser stores this cookie and sends it with subsequent requests.

For example:

Browser



The important point is that the **cookie normally contains the session identifier, not all the user's session data**.

The actual session attributes are maintained on the server.

7. Creating a Cookie Manually

Cookies can also be created directly using the Servlet API.

```
Cookie cookie = new Cookie("username", username);
```

```
cookie.setMaxAge(60 * 60);
```

```
cookie.setHttpOnly(true);
```

```
response.addCookie(cookie);
```

To retrieve it:

```
Cookie[] cookies = request.getCookies();
```

```
if (cookies != null) {
```

```
    for (Cookie cookie : cookies) {
```

```
        if (cookie.getName().equals("username")) {
```

```
            String username = cookie.getValue();
```

```
            System.out.println(username);
```

```
        }
```

```
    }
```

```
}
```

However, for authentication/session management, it is generally better to use **HttpSession** rather than storing authentication information directly in a cookie.

8. Logout Servlet

A logout operation should invalidate the server-side session.

LogoutServlet.java

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

import java.io.IOException;

@WebServlet("/LogoutServlet")
public class LogoutServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
                        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null) {
            session.invalidate();
        }

        response.sendRedirect("login.html");
    }
}
```

After:

```
session.invalidate();
```

the user's session is destroyed and protected pages should no longer be accessible using that session.

10. HttpSession vs Cookies

HttpSession

HttpSession is a **server-side session management mechanism**.

Example:

```
HttpSession session = request.getSession();  
  
session.setAttribute("username", "student");
```

The browser generally receives only a session identifier such as:

JSESSIONID=ABC123

The server uses that identifier to locate the corresponding session.

Cookies

A cookie is stored in the **client's browser**.

Example:

```
Cookie cookie =  
    new Cookie("username", "student");  
  
response.addCookie(cookie);
```

The browser sends the cookie back with matching requests.

11. Advantages of HttpSession

- Easy to implement in Servlets.
- Session data remains on the server.
- Suitable for authentication state.
- Can store multiple attributes.
- Supports session timeout.
- Can be explicitly invalidated during logout.

For example:

```
session.setAttribute("username", "student");  
session.setAttribute("role", "student");  
session.setAttribute("loggedIn", true);
```

12. Advantages and Limitations of Cookies

Advantages

- Simple client-side storage.
- Can persist between browser sessions.
- Useful for preferences and non-sensitive identifiers.
- Automatically sent with appropriate requests.

Limitations

- Stored on the client.
- Users can disable cookies.
- Cookies can be stolen if the application is insecure.
- Sensitive information should not be stored directly in cookies.

For authentication cookies, security attributes such as:

HttpOnly
Secure
SameSite

should be appropriately configured.

13. Session Timeout

A session can automatically expire after a period of inactivity.

For example:

```
session.setMaxInactiveInterval(15 * 60);
```

This sets the timeout to **15 minutes**.

The session can also be configured in `web.xml`:

```
<session-config>
  <session-timeout>15</session-timeout>
</session-config>
```

The value is in minutes.

Conclusion

A Servlet-based login system can use **HttpSession for server-side session management and cookies for carrying the session identifier** between the browser and server. After successful authentication, the server creates an `HttpSession`, stores attributes such as the username and login status, and associates the session with a `JSESSIONID` cookie. On subsequent requests, the browser sends the session ID, allowing Servlets to retrieve the user's session and maintain authentication across multiple pages.

For a login system, **HttpSession is generally preferred for storing authentication state**, while cookies are commonly used as the mechanism for transporting the session identifier. Sensitive session information should remain server-side, and HTTPS plus secure cookie settings should be used in a real application.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

A. JDBC-Based Product Management Web Application

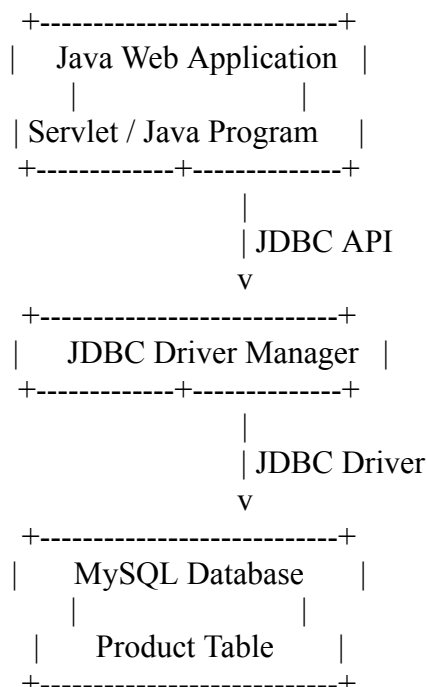
An e-commerce platform can use **JDBC (Java Database Connectivity)** to connect a Java web application to a relational database. The application can insert product details and retrieve them using SQL queries.

A typical product record contains:

- Product ID
- Product Name
- Category
- Price
- Quantity

1. JDBC Architecture

JDBC provides a standard API through which Java applications communicate with relational databases.



Main JDBC Components

1. JDBC API

Provides interfaces and classes such as:

Connection
Statement
PreparedStatement
ResultSet
DriverManager
SQLException

2. JDBC Driver

The database-specific driver translates JDBC calls into commands understood by the database.

For MySQL, the commonly used driver is **MySQL Connector/J**.

3. DriverManager

`DriverManager` manages database drivers and creates connections.

```
Connection con =  
    DriverManager.getConnection(url, username, password);
```

4. Connection

Represents a connection between the Java application and database.

5. Statement / PreparedStatement

Used to execute SQL commands.

6. ResultSet

Contains records returned by a `SELECT` query.

2. Database Design

Suppose the database is named `ecommerce`.

Create Database

```
CREATE DATABASE ecommerce;
```

Select the database:

```
USE ecommerce;
```

Create Product Table

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_name VARCHAR(100) NOT NULL,  
    category VARCHAR(50),  
    price DECIMAL(10,2),
```

```
    quantity INT  
);
```

Example data:

```
+-----+-----+-----+-----+-----+  
| product_id | product_name | category | price | quantity |  
+-----+-----+-----+-----+-----+  
| 1          | Laptop       | Electronics | 65000 | 10        |  
| 2          | Keyboard     | Accessories | 1500  | 25        |  
+-----+-----+-----+-----+-----+
```

3. JDBC Connectivity Steps

The standard JDBC process is:

1. Import JDBC packages
↓
2. Load/Register JDBC driver
↓
3. Establish database connection
↓
4. Create PreparedStatement
↓
5. Execute SQL query
↓
6. Process ResultSet
↓
7. Close resources
↓
8. Handle exceptions

4. JDBC Application

Below is a simple Java implementation that inserts and retrieves products.

ProductDAO.java

```
import java.sql.*;  
  
public class ProductDAO {  
  
    private static final String URL =  
        "jdbc:mysql://localhost:3306/ecommerce";  
  
    private static final String USER = "root";  
    private static final String PASSWORD = "root";  
  
    // Insert Product
```

```

public void insertProduct(String name,
                        String category,
                        double price,
                        int quantity) {

    String sql =
        "INSERT INTO products " +
        "(product_name, category, price, quantity) " +
        "VALUES (?, ?, ?, ?)";

    try (
        Connection con =
            DriverManager.getConnection(URL, USER, PASSWORD);

        PreparedStatement ps =
            con.prepareStatement(sql)
    ) {

        ps.setString(1, name);
        ps.setString(2, category);
        ps.setDouble(3, price);
        ps.setInt(4, quantity);

        int rows = ps.executeUpdate();

        System.out.println(
            rows + " product inserted successfully."
        );

    } catch (SQLException e) {

        System.out.println(
            "Database error while inserting product."
        );

        e.printStackTrace();
    }
}

```

```

// Retrieve Products
public void getProducts() {

```

```

    String sql =
        "SELECT product_id, product_name, " +
        "category, price, quantity " +
        "FROM products";

    try (
        Connection con =
            DriverManager.getConnection(URL, USER, PASSWORD);

```

```

        PreparedStatement ps =
            con.prepareStatement(sql);

        ResultSet rs =
            ps.executeQuery()
    ) {

        while (rs.next()) {

            int id = rs.getInt("product_id");
            String name =
                rs.getString("product_name");
            String category =
                rs.getString("category");
            double price =
                rs.getDouble("price");
            int quantity =
                rs.getInt("quantity");

            System.out.println(
                id + " | " +
                name + " | " +
                category + " | " +
                price + " | " +
                quantity
            );
        }

    } catch (SQLException e) {

        System.out.println(
            "Database error while retrieving products."
        );

        e.printStackTrace();
    }
}

```

5. Testing the Application

A main class can be used to test the DAO.

ProductApp.java

```

public class ProductApp {

    public static void main(String[] args) {

```

```

ProductDAO dao = new ProductDAO();

// Insert product
dao.insertProduct(
    "Laptop",
    "Electronics",
    65000.00,
    10
);

// Retrieve products
dao.getProducts();
}
}

```

Expected output:

1 product inserted successfully.

1 | Laptop | Electronics | 65000.0 | 10

6. Using JDBC with a Servlet

In a web application, a Servlet can receive product information from an HTML form and pass it to the DAO.

Product Form

```

<form action="ProductServlet" method="post">

    Product Name:
    <input type="text" name="name"><br><br>

    Category:
    <input type="text" name="category"><br><br>

    Price:
    <input type="number" name="price"><br><br>

    Quantity:
    <input type="number" name="quantity"><br><br>

    <input type="submit" value="Add Product">

</form>

```

ProductServlet.java

```

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;

```

```

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

@WebServlet("/ProductServlet")
public class ProductServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        String name =
            request.getParameter("name");

        String category =
            request.getParameter("category");

        double price =
            Double.parseDouble(
                request.getParameter("price")
            );

        int quantity =
            Integer.parseInt(
                request.getParameter("quantity")
            );

        ProductDAO dao = new ProductDAO();

        dao.insertProduct(
            name,
            category,
            price,
            quantity
        );

        response.setContentType("text/html");

        response.getWriter().println(
            "<h2>Product added successfully</h2>"
        );
    }
}

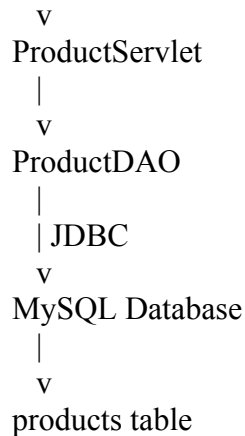
```

The architecture becomes:

```

Browser
|
| Product Form

```



7. Why Use PreparedStatement?

Instead of:

```
String sql =  
    "INSERT INTO products VALUES ("  
    + name + ", " + category + ")";
```

we use:

```
String sql =  
    "INSERT INTO products " +  
    "(product_name, category, price, quantity) " +  
    "VALUES (?, ?, ?, ?)";
```

Then:

```
ps.setString(1, name);  
ps.setString(2, category);  
ps.setDouble(3, price);  
ps.setInt(4, quantity);
```

PreparedStatement is preferred because it:

- Separates SQL from user-provided values.
- Helps prevent SQL injection.
- Handles parameter types correctly.
- Can improve performance when statements are reused.

8. Exception Handling in JDBC

Database operations can fail for many reasons:

- Database server is unavailable.
- Incorrect username/password.
- Invalid SQL query.

- Table does not exist.
- Network failure.
- Constraint violation.
- Invalid input.

JDBC uses:

SQLException

to represent database-related errors.

Example:

```
try {
    Connection con =
        DriverManager.getConnection(
            URL, USER, PASSWORD
        );

} catch (SQLException e) {

    System.out.println(
        "Unable to connect to database"
    );

    e.printStackTrace();
}
```

A better approach is to use try-with-resources:

```
try (
    Connection con =
        DriverManager.getConnection(URL, USER, PASSWORD);

    PreparedStatement ps =
        con.prepareStatement(sql)
) {

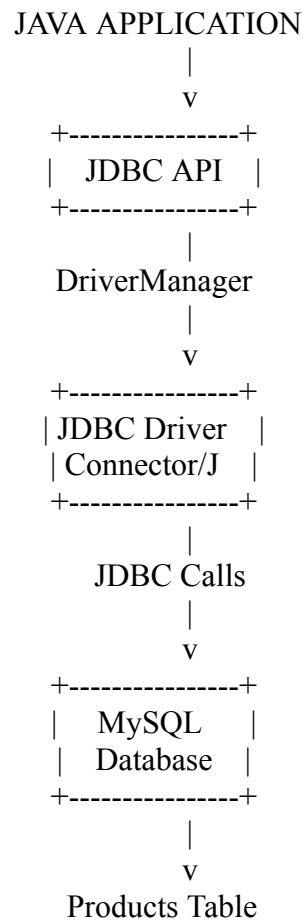
    // Database operation

} catch (SQLException e) {

    System.err.println(
        "Database operation failed: "
        + e.getMessage()
    );
}
```

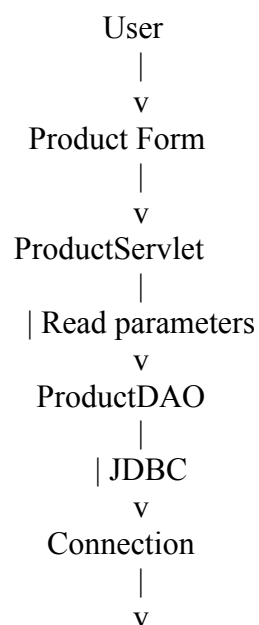
The resources are automatically closed even when an exception occurs.

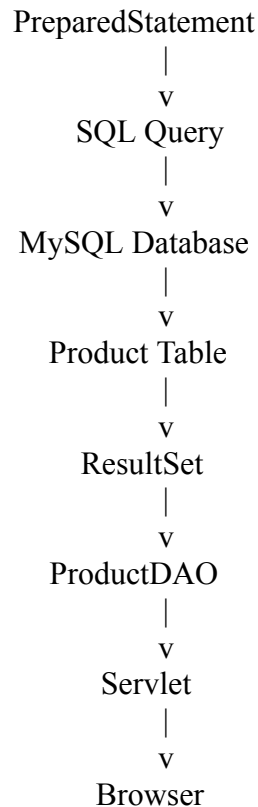
10. JDBC Architecture in Detail



The JDBC driver acts as the bridge between the Java application and the specific database system.

11. Complete Working Flow





Conclusion

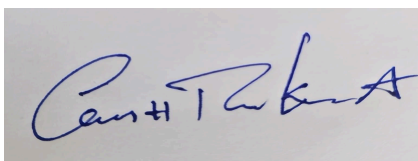
A JDBC-based e-commerce application provides a standard mechanism for Java web applications to communicate with relational databases. The main steps are **loading the driver, establishing a connection, creating a statement, executing SQL, processing the result, and closing resources**. PreparedStatement should be preferred over dynamically constructed SQL because it provides safer parameter handling and helps protect against SQL injection. Proper SQLException handling and try-with-resources ensure that database failures are handled safely and that connections and other JDBC resources are released correctly.

Code of Conduct:

I Abbui Ganesh Ram Kamal 192311183 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand